TECHNICAL REPORT · WAVEFORMRS

Validating, optimising and extending waveformrs against LALSimulation

The crate as it was found, the bugs that separated it from LALSimulation, the tests that now hold every model to LALSimulation 6.2.0 at rounding level, the optimisations, and new ports of IMRPhenomXAS, XHM, XP and XPHM with SpinTaylor, MSA and NNLO precession angles.

BRANCH	REFERENCE	PLATFORM	GENERATED
`xphm-spintaylor-validation`, 57 commits on `main` (`87f9d9b`)	lalsimulation 6.2.0, lal 7.7.0, GSL 2.8 (conda-forge, osx-arm64)	Apple M2 Ultra, macOS 26.6, rustc 1.86.0-nightly	17 October 2026

CONTENTS

1 Summary
2 The crate as found
3 Defects found and fixed
4 How the models are tested
5 Accuracy
6 Performance
7 New models
8 IMRPhenomXPHM with SpinTaylor angles
9 Issues found in LALSimulation
10 API review and suggested changes
11 Limitations and future work
12 Reproducing the results
13 Commit history

1 Summary

The crate started with two models, TaylorF2 and IMRPhenomD. It had no tests, and both models disagreed with LALSimulation by mismatches of up to 0.14. Both now agree with LALSimulation at rounding level and run about twice as fast as before. Eight more model configurations were ported, ending with IMRPhenomXPHM using the numerical SpinTaylor precession angles. Every model is checked against LALSimulation by `cargo test` and by 2,000 random parameter draws.

AGREEMENT WITH LALSIMULATION

$\leq 4.6\times10^{-11}$

Largest relative error, $|h - h_{LAL}| / \max |h_{LAL}|$, over all 2,000 random draws. The largest mismatch is 4.4×10^{-16}.

REFERENCE CASES IN `CARGO TEST`

148

In 18 files generated with lalsimulation 6.2.0, checked to ≤ 1e-9 relative error.

IMRPHENOMXPHM, SPINTAYLOR ANGLES

3.50 ms vs 8.45 ms

Rust against LALSimulation on one thread, for 5921 frequencies (2.4× faster). On a uniform grid with LAL's default multibanding, LAL takes 6.21 ms and Rust 3.52 ms.

ORIGINAL MODELS

2.1× / 2.0×

Speed-up of TaylorF2 and IMRPhenomD over the original `main`, on the same 5921-point grid.

MODEL	STATUS	DRAWS	MAX REL. ERROR	MAX MISMATCH	RUST	LAL	SPEED-UP
TaylorF2	fixed	200/200	2.2×10^{-11}	3.3×10^{-16}	111 µs	172 µs	1.5×
IMRPhenomD	fixed	200/200	2.2×10^{-11}	4.4×10^{-16}	140 µs	306 µs	2.2×
IMRPhenomXAS	new	200/200	4.6×10^{-11}	3.3×10^{-16}	103 µs	209 µs	2.0×
IMRPhenomXHM	new	200/200	2.5×10^{-15}	3.3×10^{-16}	1.01 ms	1.72 ms	1.7×
IMRPhenomXP, NNLO (102)	new	200/200	1.6×10^{-15}	4.4×10^{-16}	350 µs	658 µs	1.9×
IMRPhenomXP, MSA (223)	new	200/200	2.2×10^{-15}	3.3×10^{-16}	1.13 ms	1.56 ms	1.4×
IMRPhenomXP, SpinTaylor (320)	new	200/200	1.6×10^{-15}	3.3×10^{-16}	793 µs	2.88 ms	3.6×
IMRPhenomXPHM, NNLO (102)	new	200/200	1.5×10^{-15}	3.3×10^{-16}	2.01 ms	4.18 ms	2.1×
IMRPhenomXPHM, MSA (223)	new	200/200	2.1×10^{-15}	2.2×10^{-16}	5.00 ms	8.57 ms	1.7×
IMRPhenomXPHM, SpinTaylor (320)	new	200/200	1.4×10^{-15}	3.3×10^{-16}	3.50 ms	8.45 ms	2.4×

Accuracy from 200 random draws per model. Timings are for construction plus polarizations on 5921 frequencies (20–1500 Hz, $\Delta f = 0.25$ Hz), single thread, against `SimInspiralChooseFDWaveformSequence` with multibanding off. The aligned-spin models use $M = 60\ M_\odot$, $q = 0.5$, $\chi = (0.3, -0.2)$. The precessing models use $m_1 = 40$, $m_2 = 20\ M_\odot$, $\chi_1 = (0.4, -0.3, 0.2)$, $\chi_2 = (0.1, 0.5, -0.3)$.

The four tasks and their outcomes:

- **Assessment.** Section 2 describes the crate as found. The defects that separated it from LALSimulation are listed in section 3.
- **Testing against LALSimulation.** Every model, including each precession-angle option and each fallback path, has reference data from LALSimulation in `tests/data`. A harness compares the models over random draws (section 4, section 5).
- **Optimisation.** Every model runs faster than LALSimulation on one thread, without changing the existing API (section 6). Section 10 lists API changes that would give more.
- **IMRPhenomXPHM with SpinTaylor angles.** This is implemented, together with the models and integrator it builds on (section 7, section 8). The SpinTaylor Euler angles agree with LALSimulation's bit for bit, and the polarizations agree to ≤ 1e-15.

> **LALSimulation bug found along the way.** In multi-threaded builds, IMRPhenomXP's default MSA angles and its PNR angles are wrong at some frequencies, and the output changes from run to run. Errors reach several percent of the peak amplitude, and are of order one for PNR. The cause is a data race on `IMRPhenomXPrecessionStruct` inside an OpenMP loop. Section 9 summarises it. The full write-up with the lines to fix is `report/lalsimulation-openmp-race.md`.

2 The crate as found

On `main` (`87f9d9b`, a single commit) the crate had about 1,600 lines of Rust in 13 files:

- A **`Waveform` trait** with `orbital_speed`, `phase` and `amplitude`, plus default `waveform_single_frequency`, `waveform` and `waveform_modes` methods. The last one returns h_+ and $h_\times$ for an inclination.
- **TaylorF2**: post-Newtonian phasing up to 3.5PN with spin-orbit, spin-spin and tidal terms, and PN amplitude corrections (`src/pn.rs`, `src/taylor.rs`).
- **IMRPhenomD**: the phenomenological inspiral–merger–ringdown model, with its amplitude collocation system solved by LAPACK through `ndarray-linalg` (`src/imrphenomd/`).
- **Dependencies:** `ndarray`, `ndarray-linalg` and `blas-src` (Accelerate), with macOS-specific link flags in `.cargo/config.toml`. `autodiff` was listed but unused.
- **Tests:** none, and no reference data or comparison against LALSimulation. Two examples printed timings.

To measure where it stood, I compared the original code with LALSimulation on 100 random aligned-spin binaries per model. The match is maximised over time and phase, so differing phase conventions don't count against it. The mismatch is weighted by the aLIGO design sensitivity.

ORIGINAL MODEL	MEDIAN MISMATCH	90TH PERCENTILE	MAX MISMATCH	DRAWS WITH MISMATCH > 1E-3	MAX AMPLITUDE ERROR	TIME (5921 FREQUENCIES)
TaylorF2	5.7×10^{-4}	9.5×10^{-3}	1.4×10^{-1}	40 / 100	1.3×10^{-15}	235 µs
IMRPhenomD	4.1×10^{-4}	4.3×10^{-3}	9.7×10^{-3}	30 / 100	2.6×10^{-1}	274 µs

100 draws per model: $M \in [5, 200]\ M_\odot$ log-uniform, $q \in [1/20, 1]$ (1/18 for IMRPhenomD), $\chi_i \in [-0.9, 0.9]$, 20 Hz up to the model's cutoff, $\Delta f = 0.125$ Hz. Script: `report/baseline_accuracy.py`.

Mismatches of 1e-3 to 1e-1 are large enough to bias parameter estimation for loud signals, so the original crate couldn't stand in for LALSimulation.

3 Defects found and fixed

Comparing the coefficients one by one with LALSimulation turned up these defects. Each fix was checked against reference waveforms.

WHERE	DEFECT	EFFECT	FIX
TaylorF2 phase, `pn::taylor_3pn_ss`	The 3PN spin–spin term $\chi_1\chi_2$ was missing a factor of η.	Wrong phase for every binary with two spinning bodies. This drives the TaylorF2 mismatches of up to 0.14.	`62ff04a`
PN amplitude coefficients, `pn::taylor_f2_amplitude_*`	Used $q = m_2/m_1$ where $\delta = (m_1 - m_2)/M$ is needed.	Wrong amplitude corrections for unequal masses.	`62ff04a`
IMRPhenomD inspiral amplitude	Used the generic PN amplitude series instead of the prefactors IMRPhenomD was calibrated with (`init_amp_ins_prefactors`), whose 2.5PN and 3PN terms differ.	Inspiral amplitude errors, and through the collocation system, errors in the intermediate amplitude.	`7cfa480`
IMRPhenomD ringdown and damping frequencies	Used rational fits in the final spin. LALSimulation interpolates tabulated quasi-normal-mode data with a natural cubic spline and divides by $1 - E_{rad}$.	Shifted merger–ringdown, with amplitude errors of up to 26%.	`7cfa480` : tabulated QNM data and a port of `gsl_interp_cspline`
Reference phase and frequency	There was no way to set the reference frequency, so outputs couldn't be lined up with LALSimulation's `f_ref` / `phi_ref` .	Phase and time conventions differed from LAL.	New builder `with_reference_frequency` on both models, with existing behaviour unchanged by default.

The fixes remove all of the measured discrepancy. After them both models agree with LALSimulation to ~1e-11 (section 5). The LAPACK dependency was replaced by a 5×5 Gaussian elimination, which removes the platform-specific BLAS linking.

4 How the models are tested

Reference data in `cargo test`

`scripts/generate_reference_data.py` calls lalsimulation and stores waveforms as JSON in `tests/data` ; `serde_json` is built with `float_roundtrip` so no bits are lost. The waveforms come from `SimInspiralChooseFDWaveformSequence` on non-uniform grids and `SimInspiralChooseFDWaveform` on uniform grids, for both polarizations. Individual higher modes come from `SimIMRPhenomXHMFrequencySequenceOneMode` , Euler angles from `SimIMRPhenomXPSpinTaylorAngles` , and orbits from `SimInspiralSpinTaylorT4` . The cases are chosen to cover:

- equal and extreme mass ratios, down to 1/120 for the higher modes;
- spins up to 0.95;
- a $1.6 + 1.3\ M_\odot$ binary, whose inspiral phase reaches ~10^6 rad;
- heavy binaries that trigger LAL's fallbacks;
- non-default reference frequencies, uniform grids and restricted mode arrays.

The integration tests in `tests/lalsimulation.rs` and `tests/spintaylor.rs` require a relative error ≤ 1e-10 (TaylorF2, IMRPhenomD) or ≤ 1e-9 (IMRPhenomX family), and a mismatch ≤ 1e-14 or ≤ 1e-12. In practice the errors are far below these limits:

TEST	COMPARISONS	WORST REL. ERROR	WORST MISMATCH
TaylorF2 polarizations `taylorf2_matches_lalsimulation`	24	2.2×10^{-12}	3.3×10^{-16}
IMRPhenomD polarizations `imrphenomd_matches_lalsimulation`	16	7.2×10^{-12}	6.7×10^{-16}
IMRPhenomXAS polarizations `imrphenomxas_matches_lalsimulation`	16	1.4×10^{-11}	3.3×10^{-16}
IMRPhenomXHM polarizations `imrphenomxhm_matches_lalsimulation`	16	2.9×10^{-16}	1.1×10^{-16}
IMRPhenomXHM individual modes (both signs of m) `imrphenomxhm_modes_match_lalsimulation`	43	2.7×10^{-16}	1.1×10^{-16}
IMRPhenomXP, NNLO angles `imrphenomxp_nnlo_matches_lalsimulation`	16	5.3×10^{-16}	1.1×10^{-16}
IMRPhenomXP, MSA angles (incl. MSA → NNLO fallback) `imrphenomxp_msa_matches_lalsimulation`	22	4.2×10^{-16}	1.1×10^{-16}
IMRPhenomXP, SpinTaylor angles `imrphenomxp_spintaylor_matches_lalsimulation`	16	5.0×10^{-16}	4.4×10^{-16}

TEST	COMPARISONS	WORST REL. ERROR	WORST MISMATCH
IMRPhenomXP, SpinTaylor → MSA fallback `imrphenomxp_spintaylor_fallback_matches_lalsimulation`	16	8.5×10^{-16}	3.3×10^{-16}
IMRPhenomXP, uniform grids `imrphenomxp_uniform_matches_lalsimulation`	8	3.8×10^{-16}	1.1×10^{-16}
SpinTaylor Euler angles (SimIMRPhenomXPSpinTaylorAngles) `imrphenomxp_spintaylor_angles_match_lalsimulation`	4	0 `bit-identical`	–
IMRPhenomXPHM, SpinTaylor angles `imrphenomxphm_spintaylor_matches_lalsimulation`	16	4.2×10^{-16}	1.1×10^{-16}
IMRPhenomXPHM, NNLO angles `imrphenomxphm_nnlo_matches_lalsimulation`	16	4.9×10^{-16}	2.2×10^{-16}
IMRPhenomXPHM, MSA angles `imrphenomxphm_msa_matches_lalsimulation`	22	3.9×10^{-16}	2.2×10^{-16}
IMRPhenomXPHM, SpinTaylor → MSA fallback `imrphenomxphm_spintaylor_fallback_matches_lalsimulation`	16	6.4×10^{-16}	2.2×10^{-16}
IMRPhenomXPHM, uniform grids `imrphenomxphm_uniform_matches_lalsimulation`	10	4.2×10^{-16}	2.2×10^{-16}
IMRPhenomXPHM, restricted mode arrays `imrphenomxphm_mode_arrays_match_lalsimulation`	12	3.9×10^{-16}	≤ 0
SpinTaylorT4 orbit evolutions `spintaylor_t4_matches_lalsimulation`	15	0 `bit-identical`	–

Output of `cargo test --release --test lalsimulation -- --nocapture`. Each comparison is one polarization or mode of one case. A mismatch below zero means the two agree to rounding.

Random draws

`scripts/validate_against_lalsimulation.py` draws 200 binaries per model and evaluates both codes on the same uniform grid. Rust is driven through `examples/lal_validation.rs`, which reads JSON on stdin.

- **Parameters:** M log-uniform in [5, 300] $M_\odot$; q uniform in [1/20, 1] (1/18 for IMRPhenomD); aligned spins uniform in ±0.99, or isotropic spin vectors with $|\chi| \le 0.99$ for the precessing models.
- **Frequencies:** $f_{min} \in \{10, 15, 20\}$ Hz; $f_{ref} = f_{min}$, or uniform in $[f_{min}, 3\,f_{min}]$; $\Delta f = 0.25$ Hz up to the model's cutoff or 2048 Hz.
- **Extrinsic:** isotropic inclination; uniform reference phase; distance log-uniform in [100, 3000] Mpc.

Each draw records three things:

- the largest relative error;
- the noise-weighted mismatch (aLIGO zero-detuned high-power PSD), with *no* maximisation over time or phase, so a constant phase offset would show up;
- the median time of five calls for each code.

When LALSimulation silently changes method, the harness records the case separately and compares it against the method LAL fell back to, as in the SpinTaylor → MSA case (section 9). In the final run every draw was compared directly.

Agreement to the last bit

Matching to ~1e-9 needs the same formulas. For three components, matching better than that needs the same floating-point operations:

- **The SpinTaylor integrator.** Its adaptive step-size control turns last-bit differences in the derivatives into different accepted steps, and those grow to ~1e-7 in the spins.
- **The IMRPhenomXHM (3,2) mode.** Its intermediate phase uses a finite-difference second derivative with a step of 1e-7, which amplifies rounding by ~10^{14}. Even LALSimulation's own (3,2) mode changes by up to 3e-3 when the inputs move by 1e-15.
- **The cubic splines and final-state fits** that set the ringdown frequencies, and through them the SpinTaylor time step.

On Apple silicon, clang compiles LALSimulation and GSL with `-ffp-contract=on`. It fuses `a*b + c` into a single-rounding fused multiply-add wherever the product and the sum appear in the same expression, and OpenBLAS (behind GSL's LU solves) fuses as well. `src/spintaylor/fp.rs` provides `fma`, `fms` and `fnms` helpers that reproduce clang's contraction rule, and the ported code uses them operation by operation. The parameter-space fits of IMRPhenomXAS and XHM were generated from the C sources with the same rule. With this the SpinTaylor orbit, the splines, the remnant fits, every IMRPhenomX/XHM coefficient and the SpinTaylor and MSA Euler angles are bit-identical to LALSimulation on this platform.

On x86-64, builds without hardware FMA don't contract, and the helpers fall back to separate rounding. That configuration hasn't been tested (section 11).

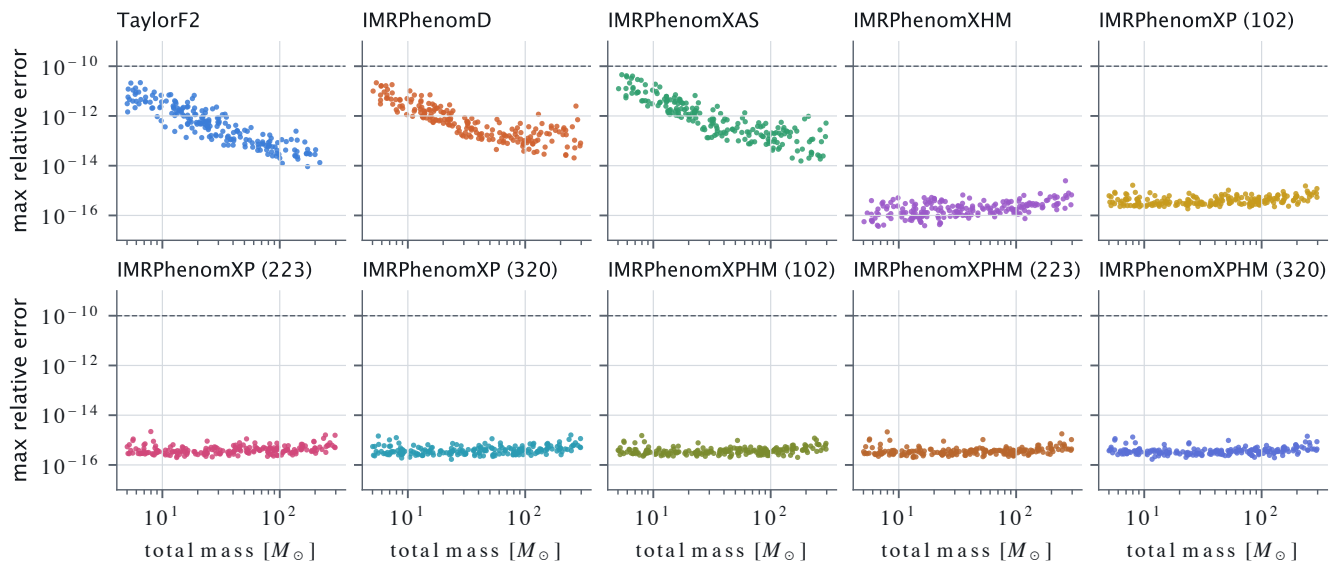
Threads

All LALSimulation runs use `OMP_NUM_THREADS=1`. Besides giving fair single-core timings, this is needed because multi-threaded IMRPhenomXP is not deterministic with the MSA angles (section 9).

5 Accuracy

MODEL	DRAWS	MAX REL. ERROR	MEDIAN REL. ERROR	MAX MISMATCH	MEDIAN MISMATCH
TaylorF2	200/200	2.2×10^{-11}	4.4×10^{-13}	3.3×10^{-16}	0
IMRPhenomD	200/200	2.2×10^{-11}	4.9×10^{-13}	4.4×10^{-16}	1.1×10^{-16}
IMRPhenomXAS	200/200	4.6×10^{-11}	6.5×10^{-13}	3.3×10^{-16}	0
IMRPhenomXHM	200/200	2.5×10^{-15}	1.8×10^{-16}	3.3×10^{-16}	0
IMRPhenomXP, NNLO (102)	200/200	1.6×10^{-15}	3.7×10^{-16}	4.4×10^{-16}	0
IMRPhenomXP, MSA (223)	200/200	2.2×10^{-15}	3.8×10^{-16}	3.3×10^{-16}	0
IMRPhenomXP, SpinTaylor (320)	200/200	1.6×10^{-15}	3.7×10^{-16}	3.3×10^{-16}	0
IMRPhenomXPHM, NNLO (102)	200/200	1.5×10^{-15}	3.6×10^{-16}	3.3×10^{-16}	0
IMRPhenomXPHM, MSA (223)	200/200	2.1×10^{-15}	3.5×10^{-16}	2.2×10^{-16}	0
IMRPhenomXPHM, SpinTaylor (320)	200/200	1.4×10^{-15}	3.5×10^{-16}	3.3×10^{-16}	0

200 random draws per model, against lalsimulation 6.2.0 with `OMP_NUM_THREADS=1`. Mismatches are not maximised over time or phase. Data: `report/data/validation_*.json`.



Largest relative error of each random draw against total mass. The dashed line is 1e-10. The ~1e-11 tail of TaylorF2, IMRPhenomD and IMRPhenomXAS comes from low-mass binaries. Their inspiral phase reaches 10^5–10^6 rad, where one unit in the last place is 1e-11 to 1e-10 rad, so these models agree with LAL to within a few roundings of the phase. The higher-mode and precessing models evaluate the (2,2) phase with LALSimulation's exact powers of Mf and operation order, so they agree to ~1e-15.

No draw comes near the tolerances of the integration tests, and every mismatch is at the level of double-precision rounding ($\leq 4.4\times10^{-16}$). There is a trade-off for the (2,2)-only models. IMRPhenomXAS evaluates its powers of Mf with one `cbrt`, one `ln` and multiplications rather than LAL's sequence of `pow` calls, which buys about 20% in speed for errors at the 1e-11 level.

6 Performance

What was changed

- **One set of transcendental calls per frequency.** TaylorF2 and IMRPhenomD now compute the shared powers of v or Mf once per frequency: one `cbrt`, one `ln` and one `sin_cos`. Amplitude and phase both use them, and nothing is evaluated twice through the trait's separate `phase` / `amplitude` methods (`5f426f9`).
- **Polynomial evaluation.** The PN phase polynomials use Estrin's scheme instead of Horner's. TaylorF2 has a fast path for point-particle phasing and falls back to a sparse Horner evaluation when tidal terms are present (`d44fb0f`). `powf` and `powi` were removed from hot loops.
- **No LAPACK.** The IMRPhenomD amplitude collocation uses a fixed-size Gaussian elimination, and the IMRPhenomX collocation systems use a port of GSL's LU solve. The setup costs 0.4–1 µs per waveform.
- **Buffer reuse.** The default `waveform_modes` builds h_+ in place in the waveform buffer instead of allocating three vectors.
- **Precessing models.**
  - Euler angles and twisting factors ($e^{ik\alpha}$, powers of the half angles, the ε phase) are computed once per frequency and per m′, and shared by all modes with that m′.
  - No allocation happens per mode or per frequency.
  - The SpinTaylor γ integration stops 64 knots past the last frequency it is needed for, and repeated Boole-rule sums are cached. The output is bit-identical.
  - The degenerate MSA case skips the spin-magnitude cubic.
- **Whole-program optimisation.** Adding the IMRPhenomXHM modules changed LLVM's inlining decisions and slowed IMRPhenomXAS from 112 to 151 µs, even though the XAS code was unchanged. A single codegen unit doesn't recover it; fat LTO does, bringing it to ~105 µs. The release profile now sets `lto = "fat"` (`3719da8`). Crates that depend on waveformrs should do the same.

Timings

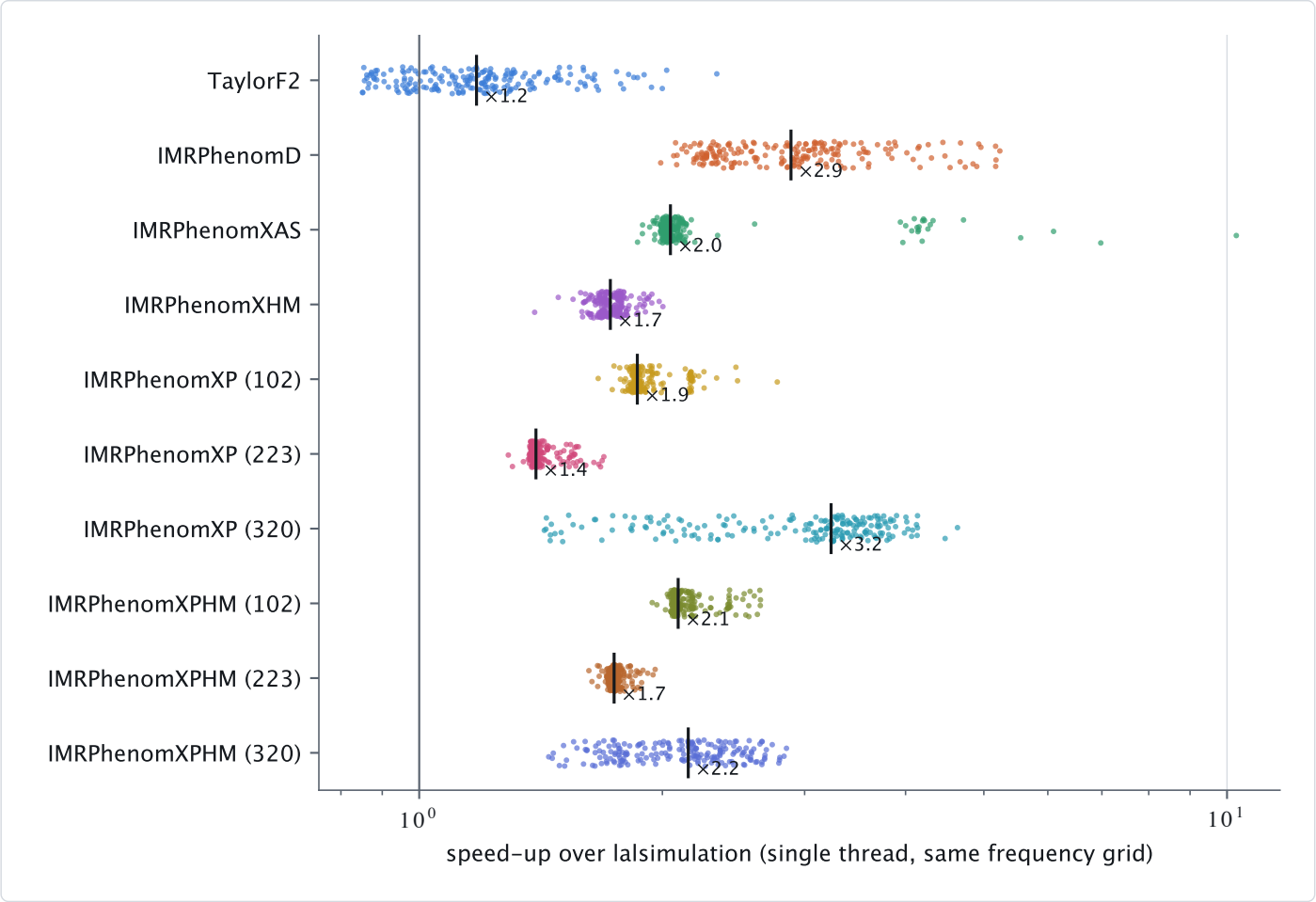
MODEL	ORIGINAL `MAIN`	RUST NOW	LAL	SPEED-UP
TaylorF2	235 µs	111 µs	172 µs	1.5×
IMRPhenomD	274 µs	140 µs	306 µs	2.2×
IMRPhenomXAS	–	103 µs	209 µs	2.0×
IMRPhenomXHM	–	1.01 ms	1.72 ms	1.7×
IMRPhenomXP, NNLO (102)	–	350 µs	658 µs	1.9×
IMRPhenomXP, MSA (223)	–	1.13 ms	1.56 ms	1.4×
IMRPhenomXP, SpinTaylor (320)	–	793 µs	2.88 ms	3.6×
IMRPhenomXPHM, NNLO (102)	–	2.01 ms	4.18 ms	2.1×
IMRPhenomXPHM, MSA (223)	–	5.00 ms	8.57 ms	1.7×
IMRPhenomXPHM, SpinTaylor (320)	–	3.50 ms	8.45 ms	2.4×

Best of 10 batches, construction plus polarizations, 5921 frequencies, single thread, against `SimInspiralChooseFDWaveformSequence`. Scripts: `examples/benchmark.rs` and `report/lal_benchmark.py`. Data: `report/data/timings.json`.

LALSimulation's higher-mode models can also *multiband* on uniform grids: they evaluate the modes on coarser grids and interpolate. This is on by default in `SimInspiralChooseFDWaveform`, which also stops at Mf = 0.3 (1015 Hz for this binary). Rust doesn't multiband. On the same uniform grid it is still faster than LAL with multibanding:

MODEL, UNIFORM GRID TO MF = 0.3	RUST	LAL, MULTIBANDING OFF	LAL, DEFAULT MULTIBANDING	SPEED-UP OVER LAL WITH MULTIBANDING
IMRPhenomXHM	681 µs	1.19 ms	1.03 ms	1.5×
IMRPhenomXPHM, SpinTaylor (320)	3.52 ms	8.42 ms	6.21 ms	1.8×

Δf = 0.25 Hz from 20 Hz, about 3,980 frequencies. IMRPhenomXHM is timed in Rust on the same frequencies with `polarizations`; IMRPhenomXPHM with `polarizations_uniform`.



Speed-up over LALSimulation for each random draw (median of five calls each, the same grid, one thread). Bars mark the medians. TaylorF2 gains least on short grids, where fixed per-call costs on both sides dominate.

Where the time goes in the precessing models

For IMRPhenomXPHM with SpinTaylor angles on the benchmark grid (about 4 ms):

- the PN integration with the ported RKF45 integrator takes about 0.6 ms;
- the γ grid (minimal-rotation integral) takes about 1.2 ms;
- the five co-precessing modes take about 0.9 ms;
- angles and twist factors at four values of m′ take about 1.0 ms;
- twisting the modes up takes about 0.3 ms.

The MSA angles cost about 15 transcendental functions per frequency. These include a Jacobi elliptic function, ported from GSL's `gsl_sf_elljac_e` with its descending Landen transformation, and they are evaluated once per m′. MSA is therefore the slowest precessing variant in Rust, and the one with the smallest speed-up over LAL. All of these steps are bit-exact ports; changing their arithmetic would trade agreement with LAL for speed.

7 New models

IMRPhenomXPHM needs IMRPhenomXAS for the (2,2) mode, IMRPhenomXHM for the other co-precessing modes, the SpinTaylorT4 equations with LAL's GSL integrator for the angles, and IMRPhenomXP's precession set-up. Each was ported and tested on its own before the next was built on it.

COMPONENT	FOLLOWS	PUBLIC API	NOTES
IMRPhenomXAS `imrphenomx/xas.rs` and shared modules	`IMRPhenomXASGenerateFD`, `IMRPhenomXSetWaveformVariables`, the phase and amplitude fits	`IMRPhenomXAS::new(M, q, χ1, χ2, D)`, `from_components`, `with_reference_frequency`, the `Waveform` trait	Default model versions. All coefficients are bit-exact. $\varphi_c = 2\varphi_{ref}$, as in IMRPhenomD.
IMRPhenomXHM `xhm.rs`, `hm_*.rs`	`IMRPhenomXHMGenerateFDOneMode`, the 122022 amplitude and 122019 phase releases	`IMRPhenomXHM::new`, `with_mode_array`, `polarizations(f, φ_ref, ι)`, `mode(l, m, f, φ_ref)`	Modes (2,±2), (2,±1), (3,±3), (3,±2) and (4,±4), including the (3,2) spheroidal–spherical mixing. No multibanding.
SpinTaylorT4 `spintaylor/`	`XLALSimInspiralSpinTaylorT4`, GSL's `rkf45` with standard control, and LAL's adaptive driver with Hermite resampling	`SpinTaylorT4::new`, `evolve_orbit(Δt, f_start, f_end)`	Bit-identical orbits, 3.3–4.5× faster than LAL.
IMRPhenomXP `xp.rs`, `precession.rs`, `spintaylor_angles.rs`, `msa.rs`, `wigner.rs`	`IMRPhenomXPGenerateFD`, `IMRPhenomXGetAndSetPrecessionVariables`, the NNLO, MSA and SpinTaylor angle routines	`IMRPhenomXP::new(&PrecessingBinary)`, `with_version`, `polarizations`, `polarizations_uniform`, `spin_taylor_angles`	PrecVersion 102, 223 and 320 (the crate default), with LAL's fallbacks.
IMRPhenomXPHM `xphm.rs`	`XLALSimIMRPhenomXPHMFrequencySequence`, `IMRPhenomXPHMTwistUp`, `IMRPhenomX_Initialize_Euler_Angles`	`IMRPhenomXPHM::new`, `with_version`, `with_mode_array`, `polarizations`, `polarizations_uniform`	No multibanding. See section 8.

Precession-angle versions and fallbacks

`PrecessionVersion` selects LAL's `PhenomXPrecVersion`:

- `Nnlo` (102): next-to-next-to-leading-order PN angles.
- `Msa` (223): the multiple-scale-analysis angles of Chatziioannou et al. (2017).
- `SpinTaylor` (320): numerical angles from integrating the SpinTaylorT4 precession equations, continued analytically through merger and ringdown.

Like LALSimulation, the code changes method without telling the caller:

- 320 uses the NNLO angles when the in-plane spins are below 1e-7.
- 320 uses MSA when the SpinTaylor integration cannot be used. This happens when the backward integration is shorter than one coarse step, or when the PN solution starts above f_{min}. In the random draws it hits about half of the binaries above 150 $M_\odot$.
- MSA uses NNLO when the MSA system cannot be initialised.

One default differs. In lalsimulation 6.2.0, IMRPhenomXP and XPHM default to `PhenomXPrecVersion` 300, which resolves to the MSA angles (223). The crate defaults to 320, which is what was asked for. Pass `PrecessionVersion::Msa` to get LAL's default.

8 IMRPhenomXPHM with SpinTaylor angles

Usage:

```
use waveformrs::imrphenomx::{IMRPhenomXPHM, PrecessingBinary, PrecessionVersion};

let binary = PrecessingBinary {
    mass_1: 40.0, mass_2: 20.0,                // solar masses
    spin_1: [0.4, -0.3, 0.2], spin_2: [0.1, 0.5, -0.3],  // at f_ref, L along z
    luminosity_distance: 400.0,                // Mpc
    inclination: 0.4, phi_ref: 0.1, f_ref: 20.0,
};
let model = IMRPhenomXPHM::with_version(&binary, PrecessionVersion::SpinTaylor)?;
let (h_plus, h_cross) = model.polarizations(&frequencies)?;           // like ChooseFDWaveformSequence
let (h_plus, h_cross) = model.polarizations_uniform(0.25, 20.0, 1024.0)?; // like ChooseFDWaveform (no multibanding)
```

A call runs these steps in order, each a port of the LAL routine named:

1. **Waveform and precession set-up**

   The masses and spins are ordered as LAL orders them. The code then builds the aligned-spin waveform variables from the L-frame spin components and computes the precession quantities: χ_p, the J frame and the angle θ_{JN}, the polarisation rotation ζ, and spherical harmonics up to $\ell = 4$ (`IMRPhenomXGetAndSetPrecessionVariables`). XPHM uses $m'_{min} = 1$ and $m'_{max} = 4$, so the angles must cover $Mf' = 2Mf/m'$ down to $(f_{min} - 0.5\ \text{Hz})\cdot 2/4$.

2 **SpinTaylorT4 evolution**

The orbit-averaged precession equations are integrated backwards and forwards from f_{ref} on a coarse grid, then on a fine grid at high frequency. The integrator is the GSL RKF45 adaptive driver as LAL uses it, with Hermite resampling to the output step (`IMRPhenomX_InspiralAngles_SpinTaylor`).

3 **α and cos β splines, remnant spin**

L and the spins are transformed to the J frame. α (unwrapped) and cos β are splined against Mf with GSL-identical natural cubic splines. The precessing final spin is computed from the spins at the end of the integration (`PhenomXPFinalSpinMod` 4), which also updates f_{RING} and f_{DAMP} of the co-precessing modes.

4 **Merger–ringdown continuation**

Analytic forms for α and β are matched to the splines at the end of the PN solution and continued through merger and ringdown (`alphaMRD_coeff` , `betaMRD_coeff`).

5 **γ from the minimal-rotation condition**

$d\gamma/df = -\cos\beta\, d\alpha/df$ is integrated with Boole's rule on a uniform grid. Its spline is evaluated up to $\max(Mf_{max}, f_{RING} + 4 f_{DAMP})\cdot 2/m'_{min}$, and the values at the transition frequency are stored (`IMRPhenomX_Initialize_Euler_Angles`).

6 **Angles per mode**

For each $m' \in \{1, 2, 3, 4\}$, α, cos β and γ are evaluated at $Mf' = 2Mf/m'$, falling back to the MRD forms beyond the splines. The $\varepsilon = -\gamma$ phase and the Wigner half-angle powers are computed once per m' and frequency.

7 **Co-precessing modes**

The (2,2) mode comes from IMRPhenomXAS and the others from IMRPhenomXHM (`IMRPhenomXHMGenerateFDOneMode`), with the precession-modified final spin and reference phase. When $q = 1$ with equal aligned spins, the odd-m' modes are skipped, as in LAL.

8 **Twist-up and polarisation rotation**

Each mode is rotated into the inertial frame with the Wigner d-matrices for $\ell = 2, 3, 4$ and summed into h_+ and $h_\times$ (`IMRPhenomXPHMTwistUp`). The result is then rotated by 2ζ.

Validation

The XPHM tests cover:

- the SpinTaylor, NNLO and MSA versions;
- uniform grids;
- restricted mode arrays;
- eight heavy binaries confirmed to make LAL fall back from SpinTaylor to MSA.

Every suite agrees with LAL to $\leq$ 7e-16 relative error. In the random draws all 200 binaries per version are compared directly; the results are in the accuracy table. The intermediate quantities (PN solution, splines, J frame, final spin, MRD coefficients, γ) were also compared bit for bit, using a small C harness linked against `liblalsimulation` 's internal functions during development.

9 Issues found in LALSimulation

OpenMP data race in IMRPhenomXP (MSA and PNR angles)

`IMRPhenomXPGenerateFD` evaluates the waveform in a `#pragma omp parallel for` loop (`LALSimIMRPhenomX.c:2281` on master). `IMRPhenomX_Return_phi_zeta_costhetaL_MSA` writes the per-frequency spin roots and spin magnitude into the shared `pPrec` struct (`LALSimIMRPhenomX_precession.c:2262–2273`) and then reads them back; the PNR angles are passed through `pPrec` in the same way. With more than one thread, some frequencies are computed with another frequency's values. Results:

- **Default IMRPhenomXP (300, which resolves to MSA):** differences of 4–8% of the peak amplitude between runs.
- **320 falling back to MSA:** 14–21%.
- **PNR angles:** errors of order one.
- **NNLO (102), SpinTaylor (320) and IMRPhenomXPHM:** unaffected.

Single-threaded output is correct and reproducible. The bug is still present on lalsuite master (`ba91eb79ca`). The full report, with a reproducer and the lines to change, is `report/lalsimulation-openmp-race.md` (with a PDF copy), and the reproducer script is `scripts/reproduce_lal_openmp_race.py`.

Smaller findings

- **Error state left set after a successful fallback.** When the SpinTaylor integration fails through `XLAL_ERROR`, the C drivers fall back to MSA and return a valid waveform, but `xlalErrno` stays set. The Python bindings then raise "Internal function call failed" for a call that succeeded.
- **Hidden global initialisation.** `XLALSimIMRPhenomXPSpinTaylorAngles` reads the global `powers_of_lalpi`, which only the waveform drivers initialise. Called first in a fresh process, its 3PN orbital angular momentum misses the π^2 term, and the angles are off by ~1e-3.
- **Ill-conditioned (3,2) set-up.** The IMRPhenomXHM (3,2) intermediate phase is fixed by a finite-difference second derivative with step 1e-7. Moving the inputs by 1e-15 changes LAL's own (3,2) mode by up to 3e-3 and the polarizations by ~6e-5, a mismatch of ~2e-9.
- **Results depend on the platform.** LAL's arithmetic depends on how the compiler contracts multiply-adds, so SpinTaylor-based results differ between arm64 and x86-64 builds at about the 1e-7 level. This is expected from the analysis in section 4 but was not measured on x86-64.

10 API review and suggested changes

All work so far kept the existing API: the `Waveform` trait and the constructors and fields of TaylorF2 and IMRPhenomD are unchanged, and new behaviour comes through additive builders and new types. These changes would make the crate more consistent, and in several cases faster:

1. **One meaning for `phi_c`.** TaylorF2 treats `phi_c` as the orbital phase and subtracts $2\phi_c$. IMRPhenomD and IMRPhenomXAS treat it as the GW phase, so matching LAL's `phi_ref` needs `phi_c = 2 phi_ref`. The precessing models take `phi_ref` directly. Using the orbital reference phase everywhere, as LAL does, would remove a common source of factor-of-two errors.
2. **Private fields and consistent units.** `total_mass`, `luminosity_distance` and `phasing` are public and mutable. The distance is passed in Mpc but stored in metres. Because a caller can change these at any time, the models can't safely cache anything that depends on them, so the evaluators rebuild a snapshot on every call. Private fields with builder methods would allow caching, and would make it impossible to change the mass without updating the phasing.
3. **Constructors that return errors.** `IMRPhenomXAS::new` and `IMRPhenomXHM::new` panic on invalid input, while `from_components` and the precessing models return `Result`. TaylorF2 and IMRPhenomD don't check their input at all. One error type and fallible constructors throughout would suit samplers, which must reject parameters rather than abort.
4. **One polarization interface.** The aligned-spin models go through `Waveform::waveform_modes(f, phi_c, theta_jn)`. IMRPhenomXHM has `polarizations(f, phi_ref, inclination)`, and IMRPhenomXP/XPHM take everything in `PrecessingBinary`. A shared trait, with all parameters (including f_{ref}, ϕ_{ref} and ι) fixed at construction, would make the models interchangeable. It would also let each model precompute things that depend on the extrinsic parameters, such as spherical harmonics and reference-phase offsets.
5. **Writing into caller-owned buffers.** Every call allocates its output vectors. A non-breaking addition such as `polarizations_into(&self, f, hp: &mut [Complex64], hc: &mut [Complex64])` would let samplers reuse memory and batch evaluations. The gain is a few percent for the fast models, plus less allocator pressure in multi-threaded use.
6. **Separating set-up from evaluation.** As in LAL, the SpinTaylor angles depend on the frequency grid through f_{min}, and $f_{ref} = 0$ means "the first frequency". Taking f_{min} explicitly would let the integration and splines be computed once and reused across grids. This matters for heterodyned or relative-binning likelihoods and ROQ bases, which evaluate one binary on several grids.
7. **Uniform grids and multibanding.** LAL's default multibanding makes IMRPhenomXHM and XPHM 1.1–1.7× faster on uniform grids, at a controlled loss of accuracy. A uniform-grid entry point already exists for XP and XPHM (`polarizations_uniform`). Porting multibanding would make it the fast path for full-band analyses.
8. **Optional parallelism.** Apart from the SpinTaylor integration and the γ integral, every per-frequency loop is independent. A `rayon` feature, comparable to LAL's OpenMP but without the shared state, would speed up long single waveforms almost linearly. The default single-threaded behaviour would stay deterministic.
9. **Precession default.** Document the difference from LAL's default (MSA), or offer `PrecessionVersion::LalDefault`.

11 Limitations and future work

- **Multibanding is not implemented.** Rust reproduces LALSimulation with `PhenomXHMThresholdMband = PhenomXPHMThresholdMband = 0`. Comparisons with LAL's defaults on uniform grids will differ at the level of the multibanding tolerance.
- **Only the default model versions are ported.** Other `PhenomXPrecVersion` values (101, 103, 104, 220–222, 224, the PNR and antisymmetric options) and the NRTidal variants are not ported. For the MSA family, 222 differs from 223 only by raising an error instead of falling back to NNLO; 220, 221 and 224 use different expressions.
- **Only one platform was tested.** Bit-for-bit agreement was verified on arm64 macOS against conda-forge builds. The x86-64 path of the FMA helpers is based on how those builds are compiled and has not been run. If it differs, the SpinTaylor-based tests at 1e-9 may need platform-specific tolerances or reference data.

- **Only one LAL version was tested.** All reference data comes from lalsimulation 6.2.0. Future LAL changes would show up as test failures, which is intended.
- **MSA speed.** The MSA angles are only 1.4–1.8× faster than LAL, because the ported elliptic functions dominate.

12 Reproducing the results

```
# environment (conda-forge)
mamba create -n waveformrs -c conda-forge python=3.12 python-lalsimulation numpy scipy matplotlib
conda activate waveformrs

# unit and LALSimulation reference tests
cargo test --release

# regenerate reference data (single-threaded: see section 9)
OMP_NUM_THREADS=1 python scripts/generate_reference_data.py --suites imrphenomxphm_spintaylor

# random draws, timings, figures and this report
OMP_NUM_THREADS=1 python scripts/validate_against_lalsimulation.py \
    --approximants TaylorF2 IMRPhenomD IMRPhenomXAS IMRPhenomXHM IMRPhenomXP IMRPhenomXP_NNLO \
    IMRPhenomXP_MSA IMRPhenomXPHM IMRPhenomXPHM_NNLO IMRPhenomXPHM_MSA --n-draws 200 --outdir report/data
OMP_NUM_THREADS=1 python report/lal_benchmark.py
python report/make_figures.py
python report/build_report.py

# the OpenMP race in LALSimulation
python scripts/reproduce_lal_openmp_race.py
```

13 Commit history

The commits on `xphm-spintaylor-validation`, oldest first, each one a logical step. The two commits that add this report are not listed. Nothing has been pushed.

COMMIT	TYPE	SUMMARY
f940559	TEST	add lalsimulation reference data and comparison helpers
62ff04a	BUG	fix PN coefficients; add reference frequency to TaylorF2
7cfa480	BUG	make IMRPhenomD agree with lalsimulation
2e1f9f6	MAINT	add LAL physical constants used by IMRPhenomX
5f426f9	PERF	faster TaylorF2 and IMRPhenomD evaluation, drop LAPACK
d44fb0f	PERF	Estrin evaluation of the inspiral PN polynomials
4aa95fb	TEST	random-draw validation harness against lalsimulation
8ec8399	FEAT	port IMRPhenomXAS from LALSimulation
60c53bb	TEST	parse reference data with exact float round trip
2b6c7e9	TEST	compare IMRPhenomXAS against lalsimulation
be10f20	EXAMPLE	add IMRPhenomXAS timing example
71743c8	TEST	include IMRPhenomXAS in validation harness and benchmark
ebea27e	FEAT	port LAL/GSL adaptive RKF45 integrator with Hermite resampling
ff2e5fe	FEAT	add SpinTaylorT4 precessing orbit evolution
3dbb470	TEST	compare SpinTaylorT4 evolution with lalsimulation
f1f53c5	MAINT	add SpinTaylorT4 timing example
db38b62	MAINT	mirror LAL's fused multiply-adds in the spline and final-state fits
9502ee7	FEAT	port IMRPhenomXP with SpinTaylor (320) and NNLO (102) Euler angles
a139f43	TEST	compare IMRPhenomXP against lalsimulation
20189b4	EXAMPLE	add IMRPhenomXP timing example
a5838ac	TEST	include IMRPhenomXP in the random-draw validation harness
7ae6228	DOC	add figure script for the technical report

COMMIT	TYPE	SUMMARY
0c02870	DOC	write up OpenMP data race in lalsimulation IMRPhenomXP MSA/PNR angles
9043fd7	DOC	add PDF copy of the lalsimulation OpenMP race writeup
b11fa66	MAINT	make the IMRPhenomXAS coefficients bit-exact with LALSimulation
e75a0cc	FEAT	port IMRPhenomXHM from LALSimulation
e744ebc	TEST	compare IMRPhenomXHM polarizations against lalsimulation
e84a4b2	MAINT	evaluate the IMRPhenomX (2,2) phase at each frequency as LALSimulation does
3b13a50	PERF	evaluate IMRPhenomXHM modes with LAL's powers of Mf and sin_cos
b800f11	TEST	compare each IMRPhenomXHM mode against lalsimulation
36f9a1d	EXAMPLE	add IMRPhenomXHM timing example
9fa011d	TEST	include IMRPhenomXHM in the random-draw validation harness
4693b55	MAINT	fix clippy lints in the PN, IMRPhenomD and example code
3719da8	PERF	enable fat LTO in the release profile
106939d	FEAT	port the IMRPhenomXP MSA Euler angles (PrecVersion 223)
e265e62	TEST	compare IMRPhenomXP MSA angles and the SpinTaylor fallback with lalsimulation
2812650	TEST	include IMRPhenomXP MSA angles in the random-draw validation
ead6203	PERF	skip the spin-magnitude cubic for degenerate MSA roots
3649640	EXAMPLE	time IMRPhenomXP with the MSA angles
0744a5a	TEST	benchmark every model on a common grid against lalsimulation
98c4b90	TEST	measure the accuracy of the original implementation against lalsimulation
80a0a52	MAINT	mirror LAL's contractions in the NNLO angles and the XP (2,2) mode
235259f	FEAT	port IMRPhenomXPHM with SpinTaylor (320) and NNLO (102) angles
9aa922d	TEST	compare IMRPhenomXPHM against lalsimulation
521fa0f	MAINT	use LAL's powers of Mf for the (2,2) mode of IMRPhenomXP with MSA angles
3a3d325	FEAT	MSA angles and the SpinTaylor-to-MSA fallback in IMRPhenomXPHM
45eb21c	TEST	compare IMRPhenomXPHM MSA angles and the SpinTaylor fallback with lalsimulation
6f1e476	PERF	shorten the SpinTaylor gamma integration for IMRPhenomXPHM
a2d5dfa	TEST	avoid a redundant match guard in the IMRPhenomXPHM test helper
be1618c	EXAMPLE	add IMRPhenomXPHM timing example
f9d386d	TEST	include IMRPhenomXPHM in the random-draw validation harness
64b2e00	EXAMPLE	time IMRPhenomXPHM with the MSA angles
2456a3a	STY	rustfmt the lalsimulation test imports
1365c59	STY	run rustfmt
d2d062e	TEST	add IMRPhenomXPHM to the benchmark
bd90e0b	DOC	the IMRPhenomXPHM loops in lalsimulation are not parallelised
8c09959	TEST	time IMRPhenomXHM/XPHM on uniform grids against LAL multibanding